hashlock.

Security Audit

Tenex (DeFi)

October 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	14
Audit Resources	14
Dependencies	14
Severity Definitions	15
Status Definitions	16
Audit Findings	17
Centralisation	45
Conclusion	46
Our Methodology	47
Disclaimers	49
About Hashlock	50

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

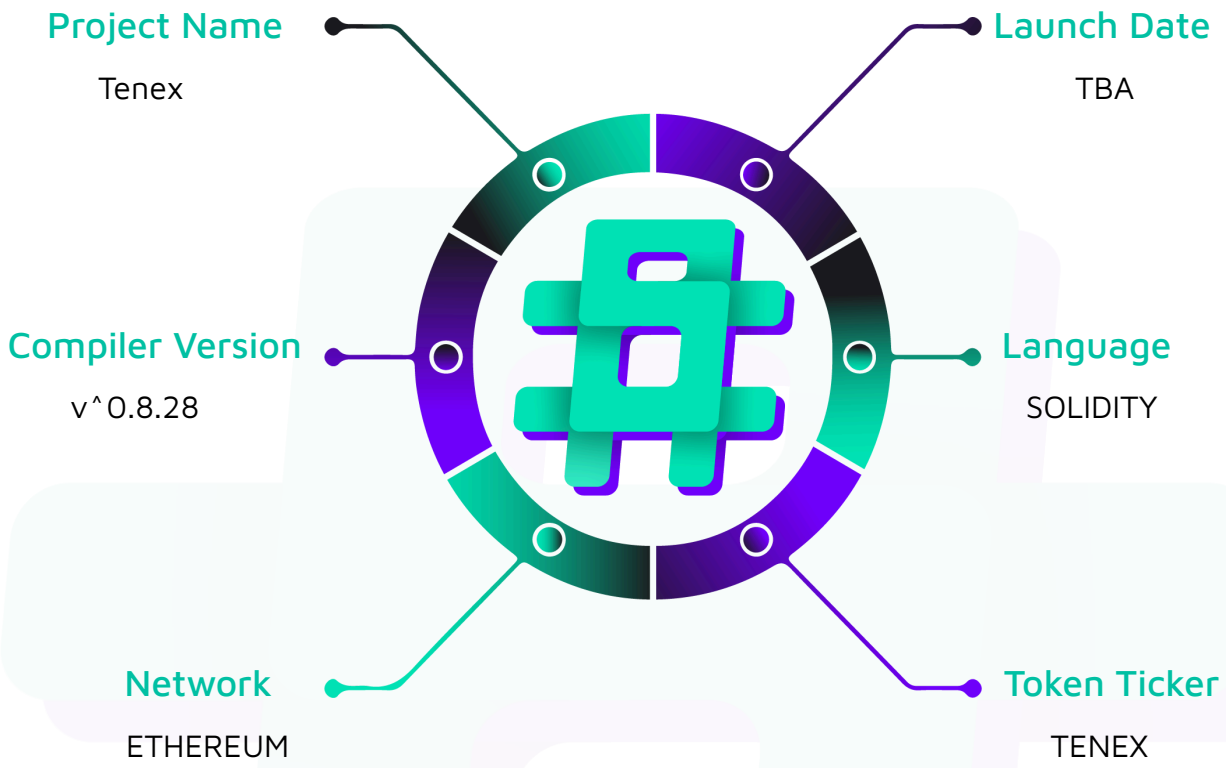
The Tenex team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

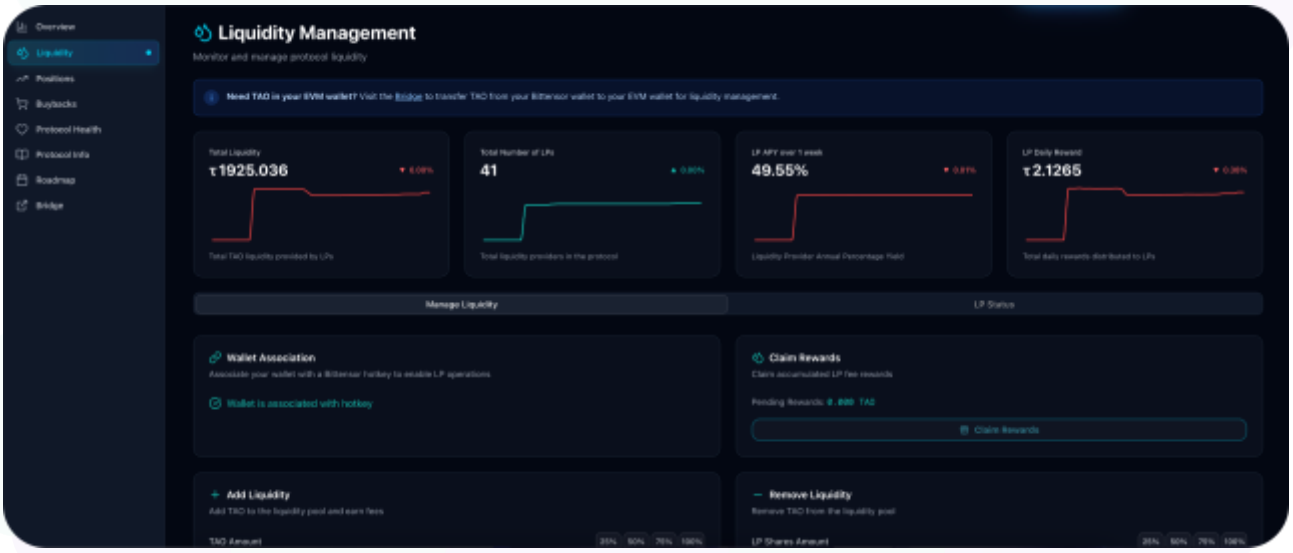
Tenexium is a decentralized long-only spot margin protocol built specifically for the Bittensor ecosystem.
Liquidity is provided exclusively in TAO, and liquidity providers earn emissions and fee shares without direct exposure to alpha volatility.
The protocol supports up to 10× leverage, enforces safety via circuit breakers, utilization caps, and dynamic fees, and allocates a portion of fees to automated token buybacks to support ecosystem sustainability.

Project Name: Tenex
Project Type: Defi
Compiler Version: ^0.8.28
Website: https://tenexium.io/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Tenex project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Tenex Smart Contracts
Platform	Ethereum / Solidity
Audit Date	October, 2025
Contract 1	TenexiumEvents.sol
Contract 2	TenexiumProtocol.sol
Contract 3	TenexiumStorage.sol
Contract 4	MultiSigWallet.sol
Contract 5	AlphaMath.sol
Contract 6	TenexiumErrors.sol
Contract 7	BuybackManager.sol
Contract 8	FeeManager.sol
Contract 9	InsuranceManager.sol
Contract 10	LiquidationManager.sol
Contract 11	LiquidityManager.sol
Contract 12	PositionManager.sol
Contract 13	PrecompileAdapter.sol
Audited GitHub Commit Hash	60245d4e24a8a9ff776815cf8d51f604d6d109c7
Fix Review GitHub Commit Hash	ea1ea6e923c9992e4de4d9a5b208214e6f493436

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section, and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

2 High severity vulnerabilities

3 Medium severity vulnerabilities

7 Low severity vulnerabilities

3 Gas Optimizations

5 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
TenexiumProtocol.sol - Allows users to: - Add TAO liquidity (payable) via addLiquidity() - Remove liquidity from pool via removeLiquidity(uint256) - Open leveraged long positions (payable) via openPosition() - Close positions and withdraw collateral via closePosition() - Add collateral to positions (payable) via addCollateral() - Liquidate undercollateralized positions via liquidatePosition() - Claim accrued LP fee rewards via claimLpFeeRewards() - Execute buyback and burn operations via executeBuyback(), executeBurn() - Associate address with a hotkey via setAssociate() - Allows admins to: - Set risk parameters and liquidation thresholds via updateFeeParameters(), updateMaxLiquidationCount(), updateAlphaPairParameters() - Update liquidity guardrails and cooldowns via updateLiquidityGuardrails(), updateActionCooldowns() - Configure buyback and insurance parameters via updateBuybackParameters(), updateInsuranceRates() - Update fee rates and fee distributions via	**Contract achieves this functionality.**

updateFeeParameters(), updateFeeDistributions() - Manage tiers and alpha pair parameters via updateTierParameters(), addAlphaPair(), removeAlphaPair(), updateAlphaPairParameters() - Update treasury, manager, conversion, insurance manager via updateTreasury(), updateManager(), updateAddressConversionContract(), updateInsuranceManager() - Withdraw protocol fees via withdrawProtocolFees() and receive insurance funds via receiveInsuranceFund() - Reset/pause circuit breaker and authorize upgrades via resetLiquidityCircuitBreaker(), UUPS _authorizeUpgrade()	
TenexiumStorage.sol - Allows users to: - Read public protocol state variables and getters (fees, tiers, utilization, pairs, LP/position mappings) - View tier, fee, and liquidity parameters - Inspect positions/LP mappings via public getters - Allows admins to: - None (storage-only; parameters set from core)	**Contract achieves this functionality.**
TenexiumEvents.sol - Allows users to: - None (events-only contract) - Allows admins to: - None	**Contract achieves this functionality.**
LiquidityManager.sol - Allows users to: - None (internal-only); add/remove liquidity exposed	**Contract achieves this functionality.**

via TenexiumProtocol - Allows admins to: - None (guardrails/cooldowns set in core)	
PositionManager.sol - Allows users to: - None (internal-only); open/close/add-collateral exposed via TenexiumProtocol - Allows admins to: - None (tier limits/permissions set in core)	**Contract achieves this functionality.**
LiquidationManager.sol - Allows users to: - None (internal-only); liquidation exposed via TenexiumProtocol - Allows admins to: - None (thresholds set in core)	**Contract achieves this functionality.**
FeeManager.sol - Allows users to: - None directly (LP reward claim routed via TenexiumProtocol) - Allows admins to: - None (fee rates/distributions set in core)	**Contract achieves this functionality.**
BuybackManager.sol - Allows users to: - None directly (buyback/burn exposed via TenexiumProtocol)	**Contract achieves this functionality.**

- Allows admins to: - None (buyback parameters set in core)	
PrecompileAdapter.sol - Allows users to: - None (internal adapter for precompile staking/unstaking/burn) - Allows admins to: - None	**Contract achieves this functionality.**
InsuranceManager.sol - Allows users to: - Send TAO to insurance fund (payable receive) - Query insurance balances via getBalance(), getNetBalance() - Allows admins to: - None (only TenexiumProtocol can call fund())	**Contract achieves this functionality.**
SubnetManager.sol - Allows users to: - None (owner-managed coordinator; accepts deposits via receive) - Allows admins to: - Set and normalize Bittensor weights via setWeights() - Update version key and Tenexium contract via setVersionKey(), setTenexiumContract() - Withdraw funds to owner via withdrawFunds() - Authorize upgrades (UUPS) via _authorizeUpgrade()	**Contract achieves this functionality.**

MultiSigWallet.sol - Allows users to: - Deposit ETH to the multisig (payable receive) - Read owners, transactions, confirmations via getters - Allows admins to: - Submit/confirm/revoke/execute transactions via submitTransaction(), confirmTransaction(), revokeConfirmation(), executeTransaction() - Add/remove owners and change lock period via addOwner(), removeOwner(), setLockPeriod() (onlySelf through executed tx) - Distribute fees to owners via distributeFees()	**Contract achieves this functionality.**
AddressConversion.sol - Allows users to: - Get ss58 pubkey for an EVM address via addressToSS58Pub() - Use blake2b hashing helpers (public view/pure functions) - Allows admins to: - None	**Contract achieves this functionality.**
AlphaMath.sol - Allows users to: - None (library; linked from other contracts) - Allows admins to: - None	**Contract achieves this functionality.**
TenexiumErrors.sol - Allows users to: - None (error namespace) - Allows admins to: - None	**Contract achieves this functionality.** **Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Tenex project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Tenex project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] LiquidationManager#_liquidatePosition - Liquidation Can Be Pushed to BadDebt by using EIP-7702 Griefing Attack

Description

A vulnerability in the liquidation process allows a user to intentionally cause the transaction to fail. This is done by exploiting a feature of `EIP-7702` that lets them temporarily add smart contract capabilities to their regular Ethereum account and reject an incoming payment.

Vulnerability Details

During liquidation, after the system sells the user's collateral to cover their debt, any leftover funds are sent back to the user's account using a low-level call. A malicious user can take advantage of `EIP-7702` to make their account behave like a smart contract that automatically rejects this incoming payment. Because the payment fails, the entire liquidation function reverts. However, the user's debt might have already been marked as "`bad debt`" by the protocol before the failure. The user can repeat this process, preventing their position from being liquidated while their debt is socialized as a loss to the protocol.

Impact

This vulnerability can lead to significant financial loss for the protocol. By preventing liquidations, the protocol is forced to absorb the user's bad debt. An attacker can keep their underwater position open longer than they should, increasing risk for the entire system. Furthermore, after forcing the protocol to take on their debt, the user might be able to withdraw their original collateral through other functions, effectively stealing funds from the protocol.

Recommendation

Instead of automatically "pushing" leftover collateral back to the user, the system should be changed to a "pull" pattern. The protocol should record the amount owed to the user in a separate balance. The user can then initiate a separate transaction to withdraw these funds at their convenience. This way, a user can't block the liquidation process, as the liquidation itself is no longer dependent on the success of returning the collateral in the same transaction.

Status

Resolved

[H-02] LiquidationManager#_liquidatePosition - Incorrect Collateral Accounting inflates System Collateral

Description

The `liquidatePosition` function in `LiquidationManager.sol` incorrectly calculates the remaining collateral in the system. When a position is liquidated, the logic only subtracts the `position.initialCollateral` from both the global `totalCollateral` and the pair-specific `pair.totalCollateral`. It fails to account for any `position.addedCollateral` that may have been added to the position during its lifetime.

Impact

This accounting error causes the system's recorded collateral to become inflated over time as liquidations occur. An inflated collateral value can mislead participants, affect protocol-wide risk calculations, and disrupt other system components that rely on this value for an accurate financial state.

Recommendation

The liquidation logic must be updated to subtract the full collateral amount of the position, which is the sum of `initialCollateral` and `addedCollateral`. This should be applied to both the global and pair-specific collateral tracking variables.Modify the relevant lines in `LiquidationManager.sol` as follows:

```
- totalCollateral = totalCollateral.safeSub(position.initialCollateral);

+     totalCollateral     =     totalCollateral.safeSub(position.initialCollateral     +
position.addedCollateral);

// ...

- pair.totalCollateral = pair.totalCollateral.safeSub(position.initialCollateral);

+   pair.totalCollateral   =   pair.totalCollateral.safeSub(position.initialCollateral   +
position.addedCollateral);
```

Status

Resolved

Medium

[M-01] BuyBackManager#_executeBuyBack - Hardcoded limit_price Causes Buyback Reverts and DoS

Description

The `buyback` flow uses a fixed `limit_price` instead of a market-derived value. If this price is below the actual rate, the precompile rejects the trade, and with `allowPartial` = false, it causes consistent reverts, resulting in a `denial-of-service`.

Vulnerability Details

`_executeBuyback` hardcodes `limit_price` to `PRECISION` (`1e9`) when calling `_stakeTaoForAlpha`. Per `IStaking.addStakeLimit`, `limit_price` is the price threshold in rao per alpha. If the actual price exceeds 1e9 rao/alpha, the precompile will not fill the order and the low-level call reverts (`TenexiumErrors.StakeFailed`). Because `allowPartial` is false, no partial fill occurs, and buybacks systematically fail. In contrast, `PositionManager` computes a dynamic limitPrice via `simSwapTaoForAlpha` and slippage tolerance; the buyback code should follow a similar approach. This mismatch makes buybacks fragile and dependent on a constant that is unlikely to match volatile on-chain prices.

```
// contracts/modules/BuybackManager.sol
function _executeBuyback() internal {
    if (!_canExecuteBuyback()) revert TenexiumErrors.BuybackConditionsNotMet();
    uint256 buybackAmount = buybackPool;
    // @audit: limit_price is hardcoded to PRECISION (1e9 rao/alpha)
    uint256 actualAlphaBought = _stakeTaoForAlpha(
        BURN_ADDRESS,
        buybackAmount,
        PRECISION,          // <- hardcoded price limit
        false,              // disallow partial
        TENEX_NETUID);
    if (actualAlphaBought == 0) revert TenexiumErrors.AmountZero();
    buybackPool -= buybackAmount;
    totalTaoUsedForBuybacks += buybackAmount;
```

```
    totalAlphaBought += actualAlphaBought;
    lastBuybackBlock = block.number;
    emit BuybackExecuted(buybackAmount, actualAlphaBought, block.number);}
```

Impact

Buybacks can fail consistently when the market price exceeds the fixed `PRECISION` threshold, preventing the protocol from converting accumulated fees into Alpha and from executing the subsequent burn. This stalls the deflationary mechanism, causes buybackPool to grow without effect, and undermines core tokenomics by failing to create buy pressure or reduce supply as designed. The denial-of-service affects protocol integrity and can persist across market conditions, impacting governance expectations and metrics that rely on successful buybacks.

Recommendation

Derive `limit_price` from a price simulation to reflect current conditions and optionally add a slippage buffer. Mirror the approach in `PositionManager` using `simSwapTaoForAlpha` and use that computed limitPrice instead of the hardcoded constant.

```
function _executeBuyback() internal {
-         uint256 actualAlphaBought = _stakeTaoForAlpha(BURN_ADDRESS, buybackAmount, PRECISION, false, TENEX_NETUID);
+    // Compute dynamic limit price from current market quote
+    uint256 expectedAlpha = ALPHA_PRECOMPILE.simSwapTaoForAlpha(
+        TENEX_NETUID,
+        uint64(buybackAmount.weiToRao()));
+    if (expectedAlpha == 0) revert TenexiumErrors.SwapSimInvalid();
+    // Optional slippage buffer can be introduced here if desired
+      uint256 limitPrice = buybackAmount / expectedAlpha; // consistent with existing pattern
+         uint256 actualAlphaBought = _stakeTaoForAlpha(BURN_ADDRESS, buybackAmount, limitPrice, false, TENEX_NETUID);
```

Status

Resolved

[M-02] FeeManager#_distributeFees - Flawed Logic Leads to Incorrect Fee Categorization

Description

In `FeeManager.sol`, a ternary operator is used to update fee totals. However, the logic is flawed: regardless of whether `isTrading` is true or false, the result of the addition is always assigned to `totalTradingFees`. When `isTrading` is `false`, `feeAmount` is correctly added to `totalBorrowingFees`, but this sum is then incorrectly assigned to `totalTradingFees`, overwriting its value and leaving `totalBorrowingFees` unchanged. This leads to the systematic miscategorization of all borrowing fees as trading fees.

Impact

Incorrect fee accounting could lead to wrong off-chain logic or execution.

Recommendation

Refactor the logic to use a standard `if/else` block to ensure that fee amounts are added to the correct state variables.

Status

Resolved

[M-03] TenexiumProtocol#setAssociate - HotKey Association Can Be Gamed, Leading to Potential DoS

Description

The `setAssociate` function allows users to associate with a hotkey but enforces a limit (`maxLiquidityProvidersPerHotkey`). A malicious actor could exploit this by creating multiple addresses and filling up the association slots for one or more critical hotkeys. This would prevent legitimate liquidity providers from associating, effectively locking them out. This could disrupt off-chain weight calculations or other protocol operations that rely on these on-chain associations.

Impact

Potential denial of service by disrupting the protocol's off-chain calculations and liquidity provider management.

Recommendation

Implement this logic directly into `_addLiquidity` so that users who staked their funds can typically be added to their Hotkeys without any issue.

Status

Acknowledged

Low

[L-01] TenexiumProtocol#distributeRewardsToUsers - Reward Distribution DoS via Malicious Receiver Revert

Vulnerability Details

The reward distribution loop sends native tokens directly to user addresses using a low-level call and reverts if any transfer fails. A malicious recipient can deliberately revert on receive to halt the entire loop. Since the function performs transfers inside a for-loop, one failing recipient prevents rewards from being distributed to all others.

```
// contracts/core/TenexiumProtocol.sol
...
(uint256 userReward = totalRewardPool.safeMul(userVolume) / totalWeeklyVolume);
if (userReward > 0) {
    (bool success,) = payable(user).call{value: userReward}("");
     if (!success) revert TenexiumErrors.TransferFailed(); // Any single failure bricks the whole distribution
}
...
```

This pattern contrasts with pull-based claims or non-reverting batch payouts, where one recipient's failure does not affect others. The function also increments currentWeek after the loop, so a revert prevents week advancement and can repeatedly block the protocol's distribution schedule.

Impact

Any single malicious or misconfigured recipient can halt the entire reward distribution, preventing all users from receiving payouts and blocking weekly progression via `currentWeek`. This degrades protocol reliability, creates user-facing delays, and can be exploited to repeatedly `DoS` reward cycles if the adversary is included in the selected recipients. While the manager curates the list (reducing likelihood), the failure mode remains systemic and can persist until governance intervenes, undermining operational continuity and user trust in rewards.

Recommendation

Do not revert on a failed per-user transfer; continue to the next recipient and optionally track failures. Alternatively, switch to a pull-based claim model where users claim their rewards to eliminate external calls in a loop. Minimal diff to avoid `DoS`:

```
function distributeRewardsToUsers(address[] calldata selectedUsers, uint16[] calldata
netUids, uint256 totalWeeklyVolume) external onlyManager nonReentrant {
- (bool success,) = payable(user).call{value: userReward}("");
-  if (!success) revert TenexiumErrors.TransferFailed();
+ (bool success,) = payable(user).call{value: userReward}("");
+ if (!success) {
+  // Skip failing recipient to avoid halting distribution
+  // Optionally: emit an event or accumulate undistributed for later handling
+ continue;
+ }
```

Status

Resolved

[L-02] TenexiumProtocol#initialize - Missing Validation Enables Unsafe Initialization

Vulnerability Details

The initialize function sets numerous critical parameters without validating ranges or invariants. Values such as `buybackRate`, fee rates, utilization thresholds, and liquidation thresholds can be set beyond safe bounds. A zero `protocolValidatorHotkey` or invalid liquidity thresholds can also be accepted. This inconsistency with the stricter update functions can start the protocol in an unsafe or inoperable state.

Recommendation

Mirror the validations enforced in update functions inside initialize: cap `buybackRate ≤ PRECISION`, enforce fee rate maximums, verify `maxUtilizationRate ≤ 95% of PRECISION`, `liquidityBufferRatio ≤ 50% of PRECISION`, and `liquidationThreshold ≥ 105% of PRECISION`. Require nonzero `protocolValidatorHotkey` and sensible minimums for thresholds (e.g., `minLiquidityThreshold ≥ 100e18`). Also validate combined share invariants (e.g., `protocolFeeGovernanceShare + protocolFeeInsuranceShare + buybackRate ≤ PRECISION`) at initialization.

Status

Resolved

[L-03] MultiSigWallet - Missing Expiration Enables Execution of Stale Transactions

Vulnerability Details

The multisig enforces only a minimum delay via `lockPeriod` but lacks any upper-bound expiration for submitted transactions. Once enough confirmations are gathered and the lock period elapses, a transaction remains executable indefinitely. This allows execution long after submission, when external conditions or protocol assumptions may have changed, undermining governance intent and risk controls.

Vulnerability Details

Submitted transactions are `timestamped` by block number at submission and can be executed once the lock period has passed, with no deadline that invalidates stale items. The relevant code shows a `lower-bound` check only:

```
// contracts/governance/MultiSigWallet.sol
mapping(uint256 => uint256) public submissionBlock; // txId => block at submission

function executeTransaction(uint256 transactionId)
    external
    nonReentrant
    onlyOwner
    txExists(transactionId)
    notExecuted(transactionId)
{
    Transaction storage txn = transactions[transactionId];
    require(block.number >= submissionBlock[transactionId] + lockPeriod, "MSW: locked");
    require(_getConfirmationCount(transactionId) >= _getThreshold(), "MSW: low conf");
    txn.executed = true;
    (bool success,) = txn.destination.call{value: txn.value}(txn.data);
    if (!success) { txn.executed = false; emit ExecutionFailure(transactionId); }
    else { emit Execution(transactionId); }
}
```

There is no check like `block.number <= deadline` or `block.timestamp <= deadline`. As a result, owners can submit/confirm a sensitive transaction and execute it at any distant

future point without re-approval, even if context (target contract logic, balances, or governance expectations) has meaningfully changed.

Impact

Indefinitely executable transactions allow owners to act on stale approvals without re-validating current conditions, enabling unintended transfers or parameter changes long after original consent. This undermines governance safeguards by decoupling execution from timely review, can enable strategic timing by a subset of owners, and increases operational risk when protocol or market state deviates from the context at submission. Even with correct thresholds and minimum delays, the absence of an expiration materially weakens the intent of time-bound, reviewed execution.

Recommendation

Introduce a per-transaction deadline at submission and enforce it at execution. Validate that the deadline is in the future and not unreasonably long. Optionally add an on-chain cancellation flow.

```
     struct Transaction {
         address destination;
         uint256 value;
         bytes data;
         bool executed;
+        uint256 deadlineBlock; // optional expiry
     }
-    function submitTransaction(address destination, uint256 value, bytes calldata data)
+     function submitTransaction(address destination, uint256 value, bytes calldata data,
uint256 deadlineBlock)
         external
         onlyOwner
         returns (uint256 transactionId)
     {
         require(destination != address(0), "MSW: dest=0");
-        transactionId = _addTransaction(destination, value, data);
+        require(deadlineBlock > block.number + lockPeriod, "MSW: bad deadline");
+        transactionId = _addTransaction(destination, value, data, deadlineBlock);
         emit Submission(transactionId);
         confirmTransaction(transactionId);
```

```
    }
-   function _addTransaction(address destination, uint256 value, bytes calldata data)
+    function _addTransaction(address destination, uint256 value, bytes calldata data,
uint256 deadlineBlock)
        internal
        returns (uint256 transactionId)
    {
        transactionId = transactionCount;
-           transactions[transactionId] = Transaction({destination: destination, value:
value, data: data, executed: false});
+        transactions[transactionId] =
+             Transaction({destination: destination, value: value, data: data, executed:
false, deadlineBlock: deadlineBlock});
        submissionBlock[transactionId] = block.number;
        transactionCount += 1;
    }
    function executeTransaction(uint256 transactionId)
        external
        nonReentrant
        onlyOwner
        txExists(transactionId)
        notExecuted(transactionId)
    {
        Transaction storage txn = transactions[transactionId];
            require(block.number >= submissionBlock[transactionId] + lockPeriod, "MSW:
locked");
+        require(block.number <= txn.deadlineBlock, "MSW: expired");
            require(_getConfirmationCount(transactionId) >= _getThreshold(), "MSW: low
conf");
        txn.executed = true;
        (bool success,) = txn.destination.call{value: txn.value}(txn.data);
        if (success) { emit Execution(transactionId); }
        else { txn.executed = false; emit ExecutionFailure(transactionId); }
    }
```

Status

Resolved

[L-04] PrecompileAdapter#_stakeTaoForAlpha - Wei-to-Rao Truncation Locks Funds and Skews Accounting

Description

The staking adapter converts `TAO` from wei (`1e-18`) to rao (`1e-9`) using integer division, which truncates remainders. When taoAmount is not an exact multiple of 1e9, the truncated remainder is not staked and remains in the contract's balance. In buyback flows, the protocol decrements buybackPool by the full wei amount even though only the rounded-down wei was actually staked. This creates locked "dust" and an accounting drift between on-chain balances and internal ledgers.

Vulnerability Details

`_stakeTaoForAlpha` computes the staking amount in rao via integer division: `uint256 amountRao = taoAmount.weiToRao();`. The conversion function divides by `1e9` with truncation so wei `% 1e9` is silently dropped:

```
library AlphaMath {
    uint256 private constant WEI_PER_RAO = 1e9;
    function weiToRao(uint256 weiAmount) internal pure returns (uint256) {
        return weiAmount / WEI_PER_RAO; // truncates remainder
    }


    function raoToWei(uint256 raoAmount) internal pure returns (uint256) {
        return raoAmount * WEI_PER_RAO;
    }
}
```

This truncated rao value is passed into the staking precompile, meaning only `raoToWei(amountRao)` wei are actually staked, while `taoAmount - raoToWei(amountRao)` wei remain in the contract. In `BuybackManager._executeBuyback`, the protocol subtracts the entire `buybackPool` from the pool after staking, even though only the rounded-down wei value can be staked due to rao granularity. As a result, funds remain idle in `address(this).balance` but are no longer tracked by `buybackPool`, leading to stranded funds and misreported totals (`e.g.`, `totalTaoUsedForBuybacks`, `totalAlphaBought`).

```
function _stakeTaoForAlpha(...) internal returns (uint256 alphaReceived) {
    ...
    uint256 amountRao = taoAmount.weiToRao(); // truncates wei remainder
    bytes memory data = abi.encodeWithSelector(
        STAKING_PRECOMPILE.addStakeLimit.selector,
        validatorHotkey,
        amountRao,
        limitPrice,
        allowPartial,
        uint256(alphaNetuid)
    );
    (bool success,) = address(STAKING_PRECOMPILE).call{gas: gasleft()}(data);
    if (!success) revert TenexiumErrors.StakeFailed();
    ...
}
```

Impact

Per call, up to `1e9-1` wei remain stranded per stake because wei-to-rao conversion truncates precision. In `buybacks` specifically, the protocol decrements `buybackPool` by the full wei amount even though only the rounded wei amount is actually staked, causing silent accounting drift and leaving untracked funds in the contract's balance. Over time, repeated operations can accumulate material dust that is not surfaced by internal accounting variables, skewing `KPIs` (`totalTaoUsedForBuybacks`, `totalAlphaBought`) and leaving protocol-owned `TAO` unusable for its intended purpose without a dedicated sweep path.

Recommendation

Always round inputs to rao granularity before staking and only decrement accounting by the actually stakeable wei. Additionally, revert if the rounded rao amount is zero to prevent 0-amount calls. For `buybacks`, compute rao first and keep the remainder in `buybackPool` so it can be used in subsequent executions.

```
function _executeBuyback() internal {
-    uint256 buybackAmount = buybackPool;
+    uint256 amountRao = buybackPool.weiToRao();
+    uint256 buybackAmount = amountRao.raoToWei(); // only stakeable wei
```

```
+    if (buybackAmount == 0) revert TenexiumErrors.AmountZero();
-         uint256 actualAlphaBought = _stakeTaoForAlpha(BURN_ADDRESS, buybackAmount,
PRECISION, false, TENEX_NETUID);
+         uint256 actualAlphaBought = _stakeTaoForAlpha(BURN_ADDRESS, buybackAmount,
PRECISION, false, TENEX_NETUID);
-    buybackPool -= buybackAmount;
+    buybackPool -= buybackAmount; // remainder stays in pool and is used next time
```

Status

Resolved

[L-05] TenexiumProtocol#distributeRewardsToUsers - Precision Loss in Pro-Rata Split Accumulates Unrecoverable Dust

Description

Reward shares are computed using integer division, which truncates fractional parts. For many recipients, the sum of truncated payouts can be materially less than the intended `totalRewardPool`. The leftover wei remain in the contract without being tracked or redistributed, leading to persistent dust accumulation. As weekly volumes and participant counts grow, unclaimed residuals increase.

Vulnerability Details

In the distribution loop, each user's payout is calculated as `totalRewardPool * userVolume / totalWeeklyVolume` with integer division, discarding fractions. The loop does not allocate or carry over the sum of truncation residuals, so the total of transfers is strictly less than or equal to `totalRewardPool`. After the loop, `currentWeek` advances and there is no mechanism to flush the remainder, leaving residual wei in contract balance. Over time, repeated rounds compound these dust amounts, especially as `selectedUsers.length` increases and users' `userVolume` values vary widely.

```
for (uint256 i = 0; i < selectedUsers.length; i++) {
    address user = selectedUsers[i];
    uint256 userVolume = userWeeklyTradingVolume[user][currentWeek];
    if (userVolume > 0) {
        // Integer division truncates fractional part
        uint256 userReward = totalRewardPool.safeMul(userVolume) / totalWeeklyVolume;
        if (userReward > 0) {
            (bool success,) = payable(user).call{value: userReward}("");
            if (!success) revert TenexiumErrors.TransferFailed();
        } } }
```

Impact

The protocol pays out less than the intended total reward due to per-user rounding, leaving leftover wei stranded in the contract balance without any accounting or sweep path. Over successive distributions with many recipients, these residuals compound, creating a growing pool of untracked funds that are not claimable by users nor

earmarked for protocol use. This degrades accounting accuracy and can distort financial metrics and expectations about total rewards distributed.

Recommendation

Combine two mitigations: (1) deterministically allocate the final remainder so the full pool is paid out, and (2) compute per-user shares with `high-precision` math to minimize rounding error. The first ensures no dust remains; the second reduces per-user truncation.

```
+import "@openzeppelin/contracts/utils/math/Math.sol";
-    for (uint256 i = 0; i < selectedUsers.length; i++) {
+    uint256 remaining = totalRewardPool;
+    for (uint256 i = 0; i + 1 < selectedUsers.length; i++) {
         address user = selectedUsers[i];
         uint256 userVolume = userWeeklyTradingVolume[user][currentWeek];
         if (userVolume > 0) {
-                        uint256 userReward = totalRewardPool.safeMul(userVolume) /
totalWeeklyVolume;
+                      uint256 userReward = Math.mulDiv(totalRewardPool, userVolume,
totalWeeklyVolume);
+            remaining = remaining.safeSub(userReward);
             if (userReward > 0) {
                 (bool success,) = payable(user).call{value: userReward}("");
                 if (!success) revert TenexiumErrors.TransferFailed();
             } } }
+    // Allocate full remainder to the last eligible recipient to guarantee no dust
+    if (selectedUsers.length > 0 && remaining > 0) {
+        address last = selectedUsers[selectedUsers.length - 1];
+        (bool ok,) = payable(last).call{value: remaining}("");
+        if (!ok) revert TenexiumErrors.TransferFailed();
+    }}
```

Status

Acknowledged

[L-06] TenexiumProtocol#distributeRewardsToUsers - Lack of Duplicate Check in Reward Distribution May Lead to Unfair Payouts

Description

The reward distribution loop in `TenexiumProtocol.sol` processes an array of `selectedUsers` provided by a manager. The loop does not check for duplicate addresses within this array. If a user's address appears multiple times in the `selectedUsers` array, they will receive a corresponding multiple of their intended reward, leading to an unfair distribution of funds.

Recommendation

Implement a mechanism to track rewarded users within the scope of a single distribution call to prevent duplicate payments. A `mapping(address => bool)` is an efficient way to achieve this.

Status

Acknowledged

[L-07] PositionManager#_openPosition - Inaccurate Entry Price Calculation Due to Use of Expected vs. Actual Values

Description

In `PositionManager.sol`, the `entryPrice` of a position is calculated using `expectedAlphaAmount`, which is the amount of tokens anticipated from a swap. However, the actual amount received, `actualAlphaReceived`, may differ due to slippage or other market dynamics. Using the expected amount results in an inaccurate `entryPrice`, which can affect the precision of `profit/loss` calculations and other position-related metrics.

Recommendation

The `entryPrice` calculation should be based on the `actualAlphaReceived` to ensure the position's data is accurate and reflects the true terms of the trade.

```
- uint256 entryPrice = (taoToStakeNet / expectedAlphaAmount).raoToWei();
+ uint256 entryPrice = (taoToStakeNet / actualAlphaReceived).raoToWei();
```

Status

Resolved

Gas

[G-01] TenexiumProtocol#distributeRewardsToUsers - Loop Gas Optimizations Missing

Vulnerability Details

The function performs two unbounded loops over `netUids` and `selectedUsers` and does not apply common gas optimizations. It repeatedly reads `selectedUsers.length` and increments `i` in a checked context, increasing gas. It also reads currentWeek from storage for each user, adding unnecessary `SLOADs`.

Recommendation

Cache `currentWeek` and array lengths in local variables before loops; use `unchecked { ++i; }` for loop increments; short-circuit early where possible. These changes reduce repeated `SLOADs` and arithmetic checks, improving gas efficiency for large batches.

Status

Resolved

[G-02] TenexiumProtocol#distributeRewardsToUsers - Repeated SS58 Conversion per Iteration

Vulnerability Details

Inside the for loop over `netUids`, the code recomputes `ADDRESS_CONVERSION_CONTRACT.addressToSS58Pub(address(this))` for every iteration. This is invariant within the function execution and results in unnecessary external calls and gas usage.

Recommendation

Compute `bytes32 protocolSs58Address = ADDRESS_CONVERSION_CONTRACT.addressToSS58Pub(address(this));` once before the loop and reuse the cached value in all iterations.

Status

Resolved

[G-03] Contracts - Multiple Gas Optimizations Opportunities Identified

Description

Several areas in the codebase could be optimized to reduce gas consumption.

1. In `AlphaMath.sol`, a multiplication function can be made more efficient by checking if either operand is zero.
2. In `FeeManager.sol`, a value `(provider.shares.safeMul(accLpFeesPerShare) / ACC_PRECISION)` is recalculated when an existing accumulated value could potentially be used.

Recommendation

Review the codebase for these and other micro-optimizations.

1. In multiplication functions, combine zero checks: `if (a == 0 || b == 0) return 0;`
2. Where possible, store and reuse intermediate calculation results instead of re-computing them within a function or across transactions.

Status

Resolved

QA

[Q-01] TenexiumStorage - Unused protocolSs58Address State Variable

Vulnerability Details

The state variable `protocolSs58Address` is declared but never set or used by the protocol logic. Calls instead compute a local `SS58` value at use sites, leaving the stored variable stale (likely zero). This creates confusion for integrators and tests that read `protocolSs58Address` expecting a meaningful, persistent value.

Recommendation

Either remove the unused state variable to reduce ambiguity, or initialize and consistently use it throughout the protocol. If keeping it, set it once (`e.g.`, in initialize) using the address conversion precompile and reference it everywhere rather than recomputing local values.

Status

Acknowledged

[Q-02] TenexiumStorage - Mutable ADDRESS_CONVERSION_CONTRACT Named Like a Constant

Vulnerability Details

`ADDRESS_CONVERSION_CONTRACT` is a mutable state variable (updated via `updateAddressConversionContract`) but is named in uppercase as if it were a constant. This naming is misleading and can cause incorrect assumptions for maintainers and integrators reading the code.

Recommendation

Rename to a non-constant style (`e.g., addressConversionContract`) and emit an event on updates to improve clarity and observability.

Status

Resolved

[Q-03] Contract - Inconsistent Solidity pragmas across repo

Vulnerability Details

Contracts use multiple compiler pragmas (`^0.8.19`, `^0.8.20`, `^0.8.24`, `^0.8.28`). This fragmentation complicates build reproducibility, tooling configuration, and audit review, and may lead to subtle differences in compilation behavior.

Recommendation

Standardize on a single compiler version across all contracts (`pin` or `caret`) and configure it in the toolchain (`Hardhat/Foundry`) with consistent optimizer settings.

Status

Resolved

[Q-04] TenexiumProtocol - No Events Emitted for Critical Parameter Changes

Vulnerability Details

Multiple `owner-only` setter functions update critical risk, fee, and buyback parameters without emitting events. This reduces transparency, hinders off-chain monitoring and auditing, and complicates incident response. Missing events on configuration mutations are a best-practice gap that can impact observability but not safety directly.

Recommendation

Emit events in all configuration updaters, including `updateRiskParameters`,`updateLiquidityGuardrails`, `updateActionCooldowns`, `updateBuybackParameters`, `updateInsuranceRates`, `updateFeeParameters`, `updateFeeDistributions`, `updateTierParameters`, `updateAlphaPairParameters`, `removeAlphaPair`, and `resetLiquidityCircuitBreaker`. Define clear, indexed event fields for changed values and actor.

Status

Acknowledged

[Q-05] TenexiumProtocol - Function Documentation is Inconsistent with Implementation

Description

The NatSpec documentation for the `removeLiquidity` function states that passing an `amount` of `0` will remove all of the user's liquidity. However, the function's implementation does not contain logic to handle this case; a `0` amount will simply do nothing. This discrepancy can mislead developers and users interacting with the contract.

Recommendation

Either update the function's documentation to accurately reflect its behavior or modify the function to implement the "`0` for all" feature as described.

Status

Resolved

Centralisation

The Tenex project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Tenex project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au